

ONLINE MASTERS IN **DATA SCIENCE**

DSC258: NATURAL LANGUAGE PROCESSING

POS TAGGING WITH HIDDEN MARKOV MODELS

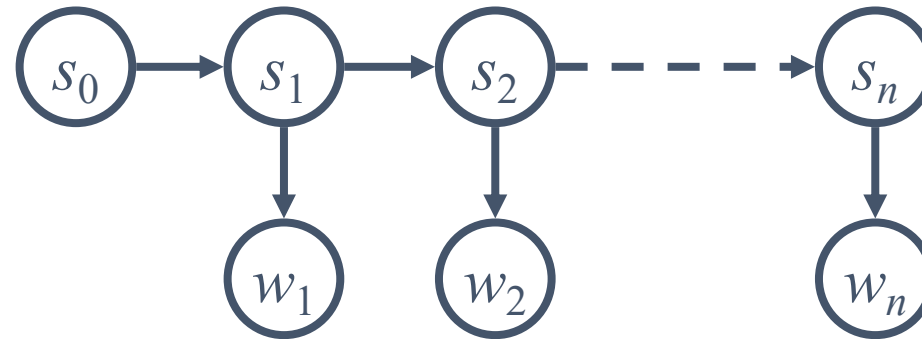
UC San Diego

COMPUTER SCIENCE & ENGINEERING
HALICIOĞLU DATA SCIENCE INSTITUTE

- Part-of-Speech Tags
- POS Tagging using spaCy
- **Classic POS Tagger using HMM**
- More POS Tagging Ideas & Summary

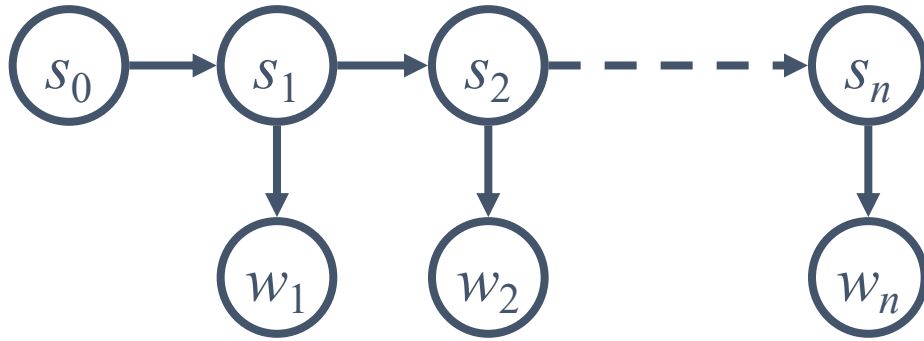
We want a model that captures:

- The (hidden) POS tag sequence **S**
- The (observed) word sequence **W**
- The dependency between adjacent words/tags (i.e., context)



$$P(\mathbf{s}, \mathbf{w}) = \prod_i P(s_i | s_{i-1}) P(w_i | s_i)$$

ASSUMPTIONS IN HMMs FOR POS TAGGING



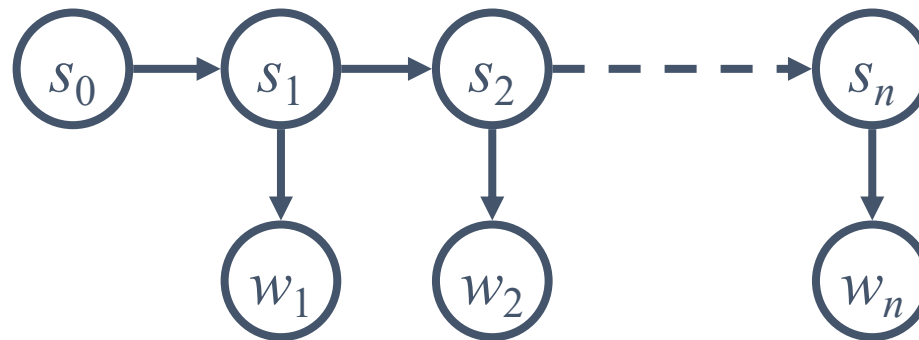
$$P(\mathbf{s}, \mathbf{w}) = \prod_i P(s_i | s_{i-1}) P(w_i | s_i)$$

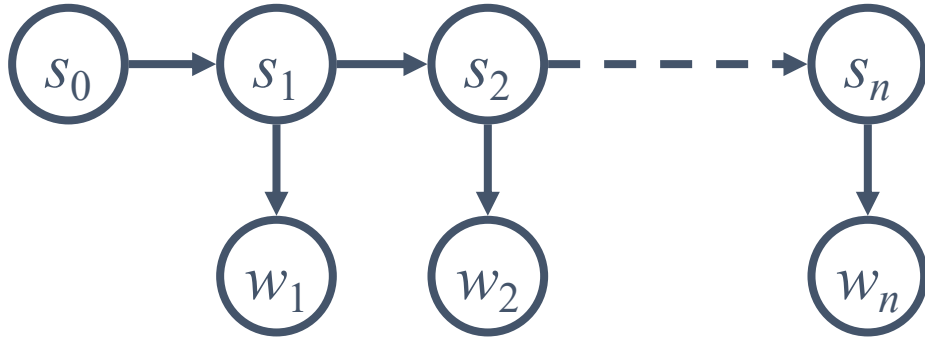
- POS tag sequence is generated by a Markov model
- (Hidden) States are POS tag n-grams
- Words are chosen independently, conditioned only on the tags/states

States encode what is relevant about the past

Transitions $P(s \mid s')$ encode well-formed tag sequences

- In a bigram tagger, states = tags 1st order
- In a trigram tagger, states = tag pairs 2nd order
-





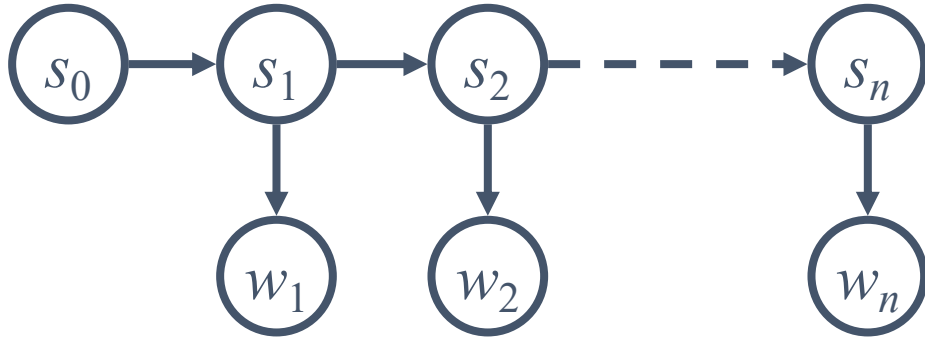
$$P(\mathbf{s}, \mathbf{w}) = \prod_i P(s_i | s_{i-1}) P(w_i | s_i)$$

When we know tag sequences, we can do counting to estimate the transition probabilities

What will happen for rare/unseen cases?

- E.g., in trigram tagger, some combinations of tag triples can be rare/unseen

- Smoothing $P(t_i | t_{i-1}, t_{i-2}) = \lambda_2 \hat{P}(t_i | t_{i-1}, t_{i-2}) + \lambda_1 \hat{P}(t_i | t_{i-1}) + (1 - \lambda_1 - \lambda_2) \hat{P}(t_i)$



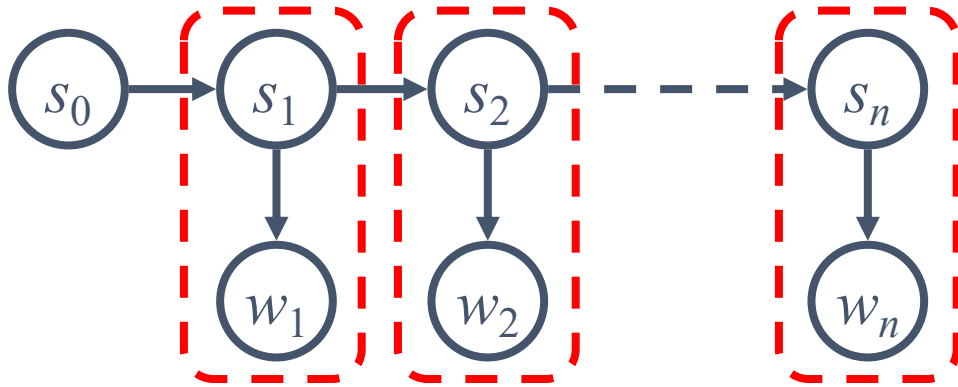
$$P(\mathbf{s}, \mathbf{w}) = \prod_i P(s_i | s_{i-1}) P(w_i | s_i)$$

When we know tag sequences, we can do counting to estimate the transition probabilities

What will happen for rare/unseen cases?

- E.g., in trigram tagger, some combinations of tag triples can be rare/unseen

- Smoothing $P(t_i | t_{i-1}, t_{i-2}) = \lambda_2 \hat{P}(t_i | t_{i-1}, t_{i-2}) + \lambda_1 \hat{P}(t_i | t_{i-1}) + (1 - \lambda_1 - \lambda_2) \hat{P}(t_i)$



$$P(\mathbf{s}, \mathbf{w}) = \prod_i P(s_i | s_{i-1}) \boxed{P(w_i | s_i)}$$

Tags are closed-world but words are open-world

- How to handle new words and new word-tag pairs?

Solutions

- `<unk>` word replacement
- Unknown word classes (e.g., affixes or word shapes)
- ...

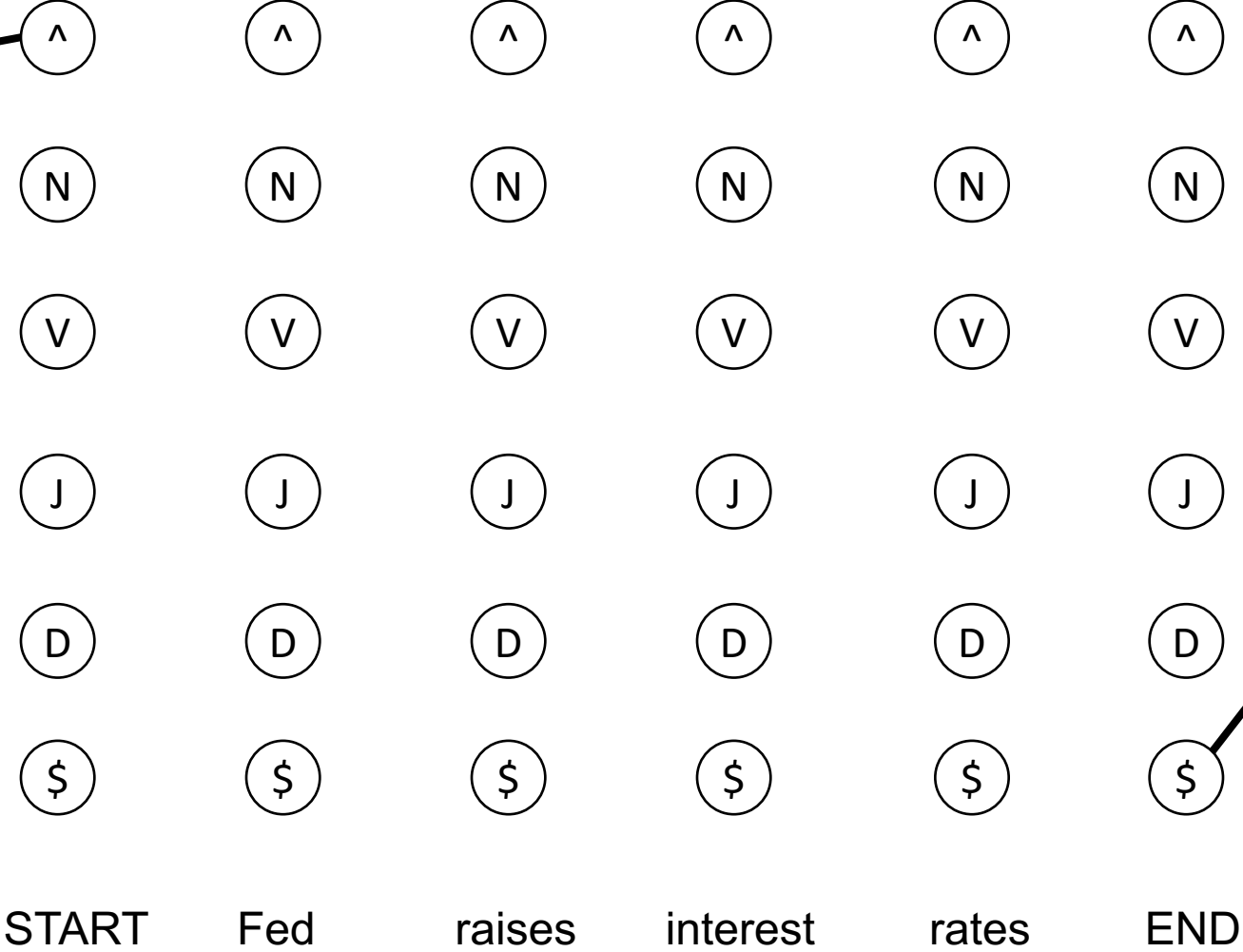
THE STATE LATTICE (COARSE POS TAGS ONLY)

Special state designed for START

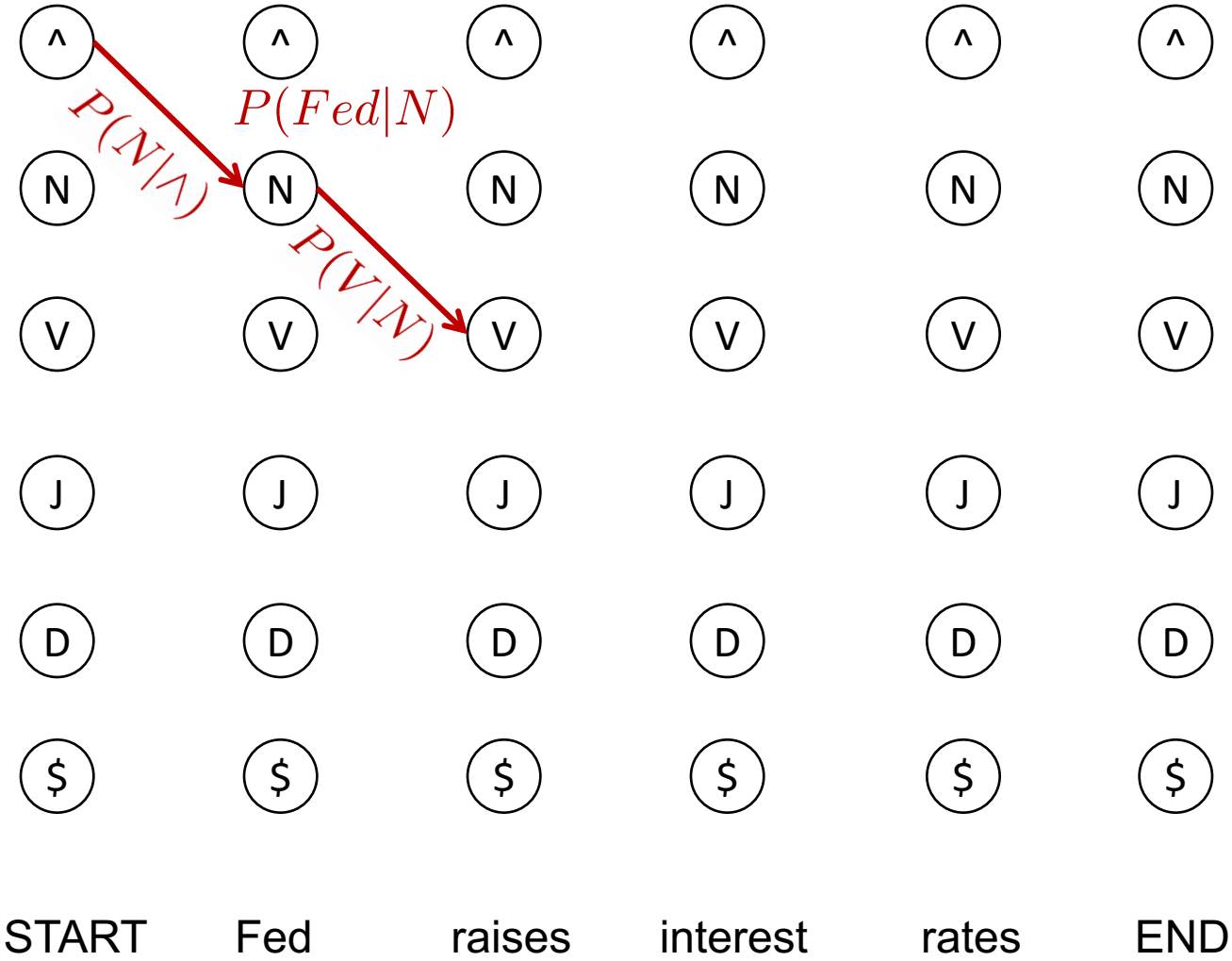
Special word added as the first token in every sentence

Special state designed for END

Special word added as the last token in every sentence



DECODING A POS TAG SEQUENCE AS A PATH IN THE STATE LATTICE



Benchmark dataset: Human annotated sentences from WSJ articles

- Every token is annotated for its POS tag
- The human annotations may have errors (~2% on this dataset)

DT NN IN NN VBD NNS VBD
The average of interbank offered rates plummeted ...

- Sometimes it's hard to annotate

JJ JJ NN
chief executive officer

NN JJ NN
chief executive officer

JJ NN NN
chief executive officer

NN NN NN
chief executive officer

Choose the most common tag for every word

- With a bad unknown word model → 90.3%
- With a carefully designed unknown word model → 93.7%

TnT (Brants, 2000)

- A carefully smoothed trigram tagger → 96.7%

State-of-the-art (SOTA) is 97+%

- Already very close to perfect
- Remember that this WSJ dataset has ~2% errors